

SBOM Manager Exploit Surface Analyzer

SBOM 목록을 실행 중인 자산, CVE 인텔리전스, 접근 가능성, TARA 위험도로 연결하는
보안 분석 프레임워크

5

자산 범주

Kernel / Daemon / Binary / Package / 3rd-party

24

검증 테스트

harness layout 기준 전체 pytest 회귀 테스트
통과

API

오케스트레이션

FastAPI 기반 scan, package, history endpoint

UI

데모

위험 대시보드, 그래프, 상세 분석, SBOM 업로드

문제: SBOM은 목록이지만, 실제 공격 표면은 관계다

사용자 스토리

- 보안 분석가**
실행 중인 서비스, 바이너리, 패키지, 취약점을 연결해서 단순 CVE 목록이 아닌 실제 공격 가능한 표면을 우선순
위화하고 싶다.
- 취약점 관리자**
외부에 열린 포트가 어떤 프로세스와 바이너리에서 왔는지 확인하고, 패치 전후 위험 변화까지 비교하고 싶다.

프로젝트 목표

- 수집: kernel, daemon, binary, package 상태를 probe로 자동 수집
- 상관분석: port -> PID -> binary -> package -> CVE 관계 구성
- 위험평가: 노출도, 완화기법, TARA feasibility를 반영
- 운영화: API, dashboard, scan history, comparison으로 반복 분석 지원



분석가 관점

서비스/바이너리/패키지를 연결해 실제 공격 경
로를 찾는다



운영자 관점

외부 노출 자산과 패치 전후 위험 변화를 비교한
다

Repo 자체를 개발 하네스로 사용: tree -L 3 구조 + 운영 문서

```
sbomManager/ # tree -L 3 -d -I '.__pycache__|node_modules|.git'
├── docs/
│   ├── architecture/
│   ├── decisions/
│   ├── requirements/
│   └── threat-model/
├── evaluations/
│   ├── benchmarks/
│   ├── reports/
│   └── slide/
├── memory/
│   └── data/
│       ├── results/
│       ├── scan_history/
│       └── test_cpe_cache/
├── plans/
│   ├── active/
│   └── completed/
├── scripts/
├── skills/
│   ├── final-presentation-builder/
│   ├── sbom-manager-analysis/
│   └── todo-tracking-reconciliation/
├── src/
│   ├── api/
│   ├── core/
│   ├── plugins/
│   ├── web/
│   └── tasks/
│       ├── doing/
│       ├── done/
│       └── todo/
│   ├── core/
│   │   ├── graph/
│   │   ├── binaries/
│   │   ├── kernel/
│   │   └── frontend/
│   ├── routes/
│   ├── daemons/
│   ├── intelligence/
│   ├── packages/
│   ├── sbom_parsers/
│   └── third_party/
```

CLAUDE.md / AGENTS.md

root와 domain별 agent operating rule. 수집 범위, graceful degradation, domain execution 기준을 고정한다.

SESSION_LOG.md

tasks/done 및 src/evaluations/memory별 변경 이력을 남겨 sprint 증거와 회고를 human-readable하게 유지한다.

progress.json

tasks/progress.json과 module-level progress.json이 active todo, status, 검증 흐름의 machine-readable source 역할을 한다.

design.md / IMPLEMENTATION.md

plugin design과 core 구현 상태를 코드 옆에 두어 architecture -> implementation trace를 유지한다.

skills/

TARA 하위 폴더를 제거하고 실제 사용 skill 3개를 skills/ 바로 아래에 배치했다.

scripts/ + evaluations/

build/test/verify/launch와 benchmarks를 분리해 실행과 검증을 반복 가능한 harness로 묶었다.

6번의 스프린트

상관분석 -> UI -> 검증까지 확장

Sprint 1. 기반 구조

- 프로젝트 vision/CLAUDE.md 작성
- harness layout 초안 확정
- Pydantic 기반 asset schema 설계

Sprint 2. 시스템 Probe

- kernel 정보 수집
- daemon port -> PID 매핑
- binary ELF mitigation 분석

Sprint 3. Intelligence

- CPE resolver 추가
- NVD CVE provider 연결
- OSV/PURL 기반 package intelligence 준비

Sprint 4. Risk & Graph

- asset relation graph 구성
- TARA Impact x Feasibility scoring
- exposure/mitigation 기반 우선순위화

Sprint 5. API & UI

- FastAPI scan endpoint
- React dashboard
- inventory/detail/attack path demo 화면

Sprint 6. 마무리 TODO

- Package Probe 구현
- Remediation Engine 구현
- Scan History/Compare API 구현
- harness repo layout 정리

각 스프린트 종료 기준: 구현 완료가 아니라 검증 로그까지 남기는 것

1. Sprint Planning

- tasks/progress.json active_todo 기준으로 scope 선택
- tasks/todo/TODO_TRACKING.md에 완료/보류 기준 명시
- 각 기능의 검증 명령을 먼저 정함

2. Daily Build/Test

- 작은 단위 구현 후 pytest 실행
- scripts/build.sh로 import/schema 오류 확인
- FastAPI TestClient로 endpoint smoke test

3. Review & Retrospective

- tasks/done/SESSION_LOG.md 기록
- 남은 TODO는 completed/deferred로 재정리

```
검증 루틴
1) 기능 단위/통합 테스트
$ ./scripts/test.sh
24 passed in 13.98s

2) 통합 회귀 검증
$ ./scripts/verify.sh
24 passed in 11.32s

3) 정적 실행 가능성 확인
$ ./scripts/build.sh
# passed

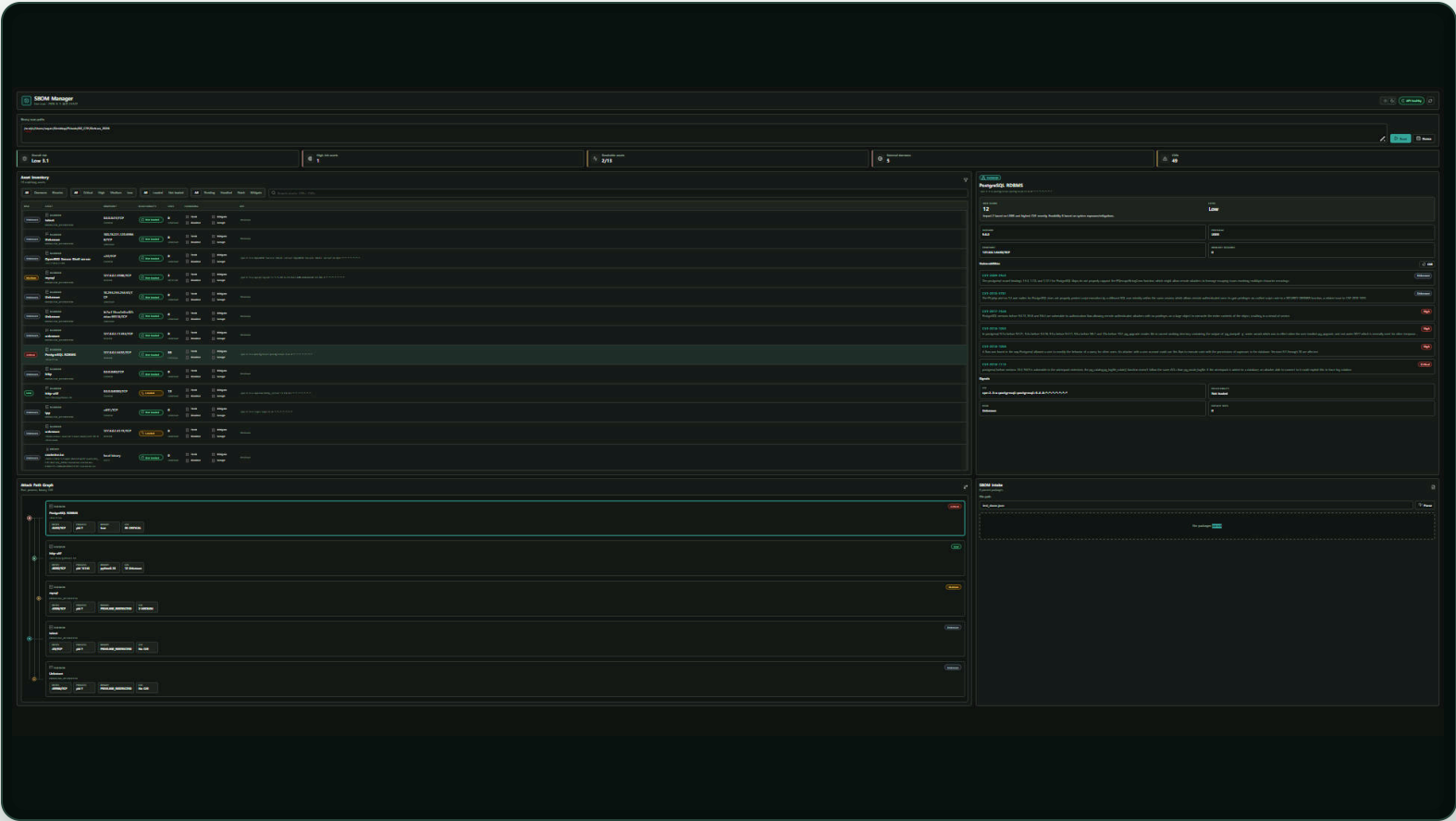
4) Full-stack launch smoke
$ ./scripts/launch.sh
Backend /health -> 200
Frontend / -> 200

5) API smoke check
POST /api/v1/packages/probe -> 200
GET /api/v1/scans -> 200
GET /api/v1/scans/compare/{base}/{target} -> 200
```



UI Dashboard

with Detailed info(cpe + cve)



전체 대시보드: scan control, summary card, asset inventory, detail panel, attack path graph, SBOM upload가 한 화면에 표시됨. ``scripts/launch.sh``로 backend/frontend를 함께 실행.

DAEMON

PostgreSQL RDBMS

cpe:2.3:a:postgresql:postgresql:9.6.0:*:*:*:*:*

RISK SCORE

12

LEVEL

Low

Impact 2 based on USER, and highest CVE severity; Feasibility 6 based on system exposure/mitigations.

VERSION

9.6.0

PRIVILEGE

USER

ENDPOINT

127.0.0.1:5432/TCP

MEMORY REGIONS

0

Vulnerabilities

CVE

CVE-2009-2943

Unknown

The postgresql-ocaml bindings 1.5.4, 1.7.0, and 1.12.1 for PostgreSQL libpq do not properly support the PQescapeStringConn function, which might all...

CVE-2010-3781

Unknown

The PL/php add-on 1.4 and earlier for PostgreSQL does not properly protect script execution by a different SQL user identity within the same session, ...

CVE-2017-7548

High

PostgreSQL versions before 9.4.13, 9.5.8 and 9.6.4 are vulnerable to authorization flaw allowing remote authenticated attackers with no privileges on a large ...

CVE-2018-1053

High

In postgresql 9.3.x before 9.3.21, 9.4.x before 9.4.16, 9.5.x before 9.5.11, 9.6.x before 9.6.7 and 10.x before 10.2, pg_upgrade creates file in current working di...

CVE-2018-1058

High

A flaw was found in the way Postgresql allowed a user to modify the behavior of a query for other users. An attacker with a user account could use this fla...

CVE-2018-1115

Critical

postgresql before versions 10.4, 9.6.9 is vulnerable in the adminpack extension, the pg_catalog.pg_logfile_rotate() function doesn't follow the same ACLs th...

Signals

CPE

cpe:2.3:a:postgresql:postgresql:9.6.0:*:*:*:*:*

REACHABILITY

Not loaded

USER

Unknown

EXPLOIT REFS

0

역할 분담과 프로젝트 회고

팀원별 기여

- **qnfdudemr**: 취약점 인텔리전스 API 담당 - CPE resolver와 CVE/NVD provider를 연결해 자산을 공개 취약점 데이터와 매핑.
- **sti09181**: Package Probe 담당 - dpkg-query와 Python dist-info를 활용해 package name, version, ecosystem, PURL 수집 로직 구현.
- **won6c**: 전체 통합 담당 - architecture, collector, kernel/daemon/binary probe, graph/risk, remediation, scan history, UI, tests, docs, final integration 수행.

Challenges -> Lessons

- **NVD API rate limit**
외부 API 의존성을 줄이기 위해 cache/offline-friendly provider가 필요함.
- **FastAPI JSON recursion**
Pydantic 모델과 jsonable encoding을 명시적으로 관리해야 함.
- **TODO drift**
progress.json을 active scope의 single source of truth로 유지해야 함.
- **CVE count 한계**
실제 위험은 CVE 개수보다 exposure, reachability, mitigation으로 결정됨.

Thank you

<https://github.com/won6c/sbomManager>